

Pel

Appunti del corso

NOTA AL LETTORE

Questi appunti raccolgono i contenuti del corso **Programmazione e laboratorio modulo 1**.

Struttura degli appunti. Il materiale è organizzato nei seguenti capitoli:

- **Capitolo 1 – Fondamenti:** variabili, tipi di dato, reference, strutture di iterazione (`for`, `while`, `do-while`), passaggio dei parametri e primi algoritmi.

Convenzioni tipografiche.

- Il codice inline è scritto in `carattere monospazio`.
- I blocchi di codice sono numerati e evidenziati.
- Le definizioni, i teoremi e le osservazioni sono racchiusi in riquadri colorati per facilitarne l'individuazione.

INDICE

Nota al lettore	3
Prefazione	3
Indice	5
1 Fondamenti	7
1 Definizioni fondamentali	8
1.1 Casting	8
1.2 Perdita di informazione:	10
1.3 Left value Right value	11
2 Passaggio dei parametri	11
3 Uso preferenziale dei cicli	14
3.1 Uso preferenziale del for	14
3.2 While	15
4 Do while	16
5 Programmazione strutturata	16
2 Vettori e stringhe	19
1 Array	20
1.1 Definizione e idea intuitiva di array	20
1.2 Limiti degli array e motivazione per strutture più astratte	20
2 Vector	21
2.1 Idea generale	21
2.2 Struttura interna	21
2.3 Metodi principali	22
2.4 Passaggio dei parametri	22
3 Bubble sort	23
3.1 Il codice completo	27
4 String	27
4.1 Differenze rispetto a <code>char*</code>	27
4.2 Input e output	27
4.3 Operazioni fondamentali	27
4.4 Concatenazione e costruzione incrementale	28
4.5 Estrazione, ricerca e sostituzione	29
5 List	30
5.1 Dichiarazione e dimensione	30
5.2 Inserimento e rimozione in testa e in coda	30

6	Problemi e pattern di programmazione	31
6.1	Pattern esistenziale: é presente?	31
6.2	Pattern universale: tutti soddisfano la proprietà?	33
6.3	Esempio di esecuzione	35
6.4	Ottimizzazione: terminazione anticipata	37
6.5	Esempio di early termination	38
3	Ricorsione	41
1	Introduzione	42
1.1	Struttura formale	42
2	Ricorsione lineare	42
2.1	Post-order	43
2.2	Pre-order	44
2.3	Tail recursion	45
3	Tail recursion	47
3.1	Senza accumulatore	47
3.2	Con accumulatore	49
3.3	Accumulatore stringa	50
3.4	Accumulatori multipli paralleli	51
4	Divide et impera	52
4.1	Divide et impera con combine esplicita	52
4.2	Divide et impera senza combina, con riduzione diretta	54
5	Backtracking	57
5.1	Backtracking su scelte binarie	57
5.2	Backtracking su permutazioni	58
5.3	Backtracking con vincoli strutturali	59
4	Classi	61
1	Introduzione alle classi	62
1.1	Metodi	62
1.2	Struttura generale di una classe	62
1.3	Implementare i metodi fuori dalla classe	62
1.4	Incapsulamento	62
1.5	Coordinate	63
1.6	Implementazione dei metodi della classe Punto	63
1.7	costruttore	64
1.8	Distruttore	65
2	Live Coding Classi in C++	65

CAPITOLO 1

FONDAMENTI

1	Definizioni fondamentali	8
1.1	Casting	8
1.2	Perdita di informazione:	10
1.3	Left value Right value	11
2	Passaggio dei parametri	11
3	Uso preferenziale dei cicli	14
3.1	Uso preferenziale del for	14
3.2	While	15
4	Do while	16
5	Programmazione strutturata	16

Introduciamo i concetti di base del linguaggio C++ da cui partirà tutto il resto del corso.

1 Definizioni fondamentali

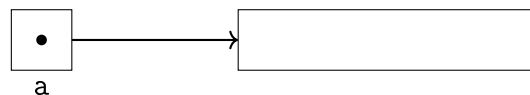
Definizione 1.1: Concetti fondamentali

- **Tipi di dati:** Un tipo di dato in c++ definisce l'insieme dei valori che può assumere e le operazioni che possono essere eseguite su di essi.
- **Oggetto:** Un oggetto è un area di memoria a cui è associato un tipo.
- **Variabile:** Una variabile è un oggetto a cui è associato un identificatore.

Un puntatore è una variabile che contiene l'indirizzo di un'area di memoria. Nell'esempio seguente, `a` è un puntatore a un intero allocato dinamicamente tramite `malloc()`:

Esempio 1.1 (Allocazione di memoria dinamica).

```
1  int *a = (int*) malloc(sizeof(int));
```



La funzione `malloc()` (ereditata dal linguaggio C) alloca un blocco di memoria della dimensione specificata da `sizeof(int)` e restituisce un puntatore generico (`void*`), che viene convertito a `int*` tramite un cast esplicito. La variabile `a` non contiene direttamente un valore intero, ma l'indirizzo della cella di memoria allocata. Per accedere al valore puntato si usa l'operatore di dereferenziazione `*a`.

1.1 Casting

Il casting consiste nella conversione di un valore da un tipo all'altro. Nel linguaggio C++, esistono due tipologie di conversione:

- **Conversione implicita:** il compilatore converte da solo i tipi nelle espressioni “miste”, promuovendo il tipo più piccolo verso quello più grande.

vediamo un esempio:

Esempio 1.2 (Casting implicito).

```
1      int a = 7;
2      int b = 3;
3      double c= a*1.0/b;  // a e b vengono promossi a
    in double => c = 2.333...
```

Per esempio `int` $\rightarrow$ `double`, in questo caso non abbiamo perdita di informazione.

- **Conversione esplicita (cast):** In questo caso viene forzata la conversione da un tipo all'altro. In C++ viene utilizzata la sintassi. `static_cast<type>(x)`.

vediamo un esempio:

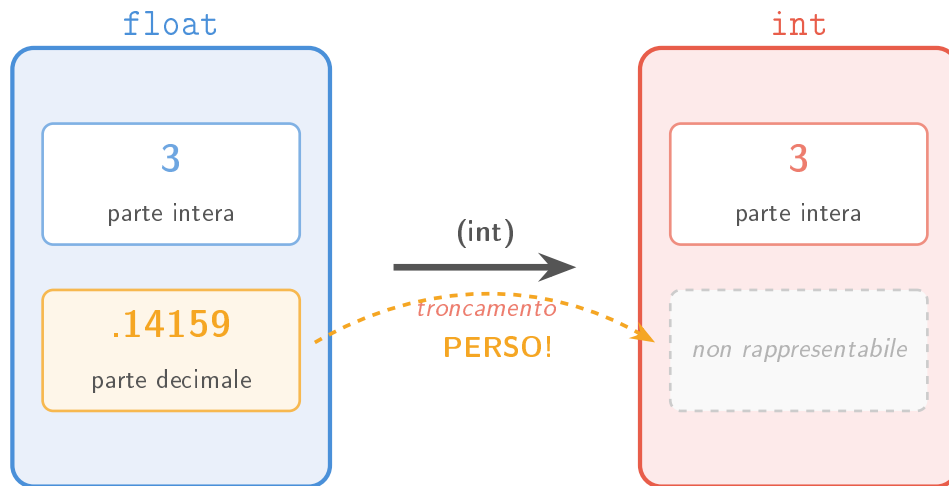
Esempio 1.3 (Casting esplicito).

```
1      int total = 5;
2      int count = 2;
3      double average;
4
5      // senza cast: promozione automatica
6      average = total / count;          // 5 / 2 = 2 (int),
    poi 2 => 2.0
7
8      // con cast: controlli tu la conversione
9      average = static_cast<double>(total) / count; //
    5.0 / 2 = 2.5
10
```

1.2 Perdita di informazione:

Definizione 1.2: Narrowing

La perdita di informazione si ha quando si converte da un tipo “capiente” a un più piccolo.



Narrowing conversion: float (32 bit, con decimali) $\rightarrow$ int (32 bit, solo interi) $\Rightarrow$ la parte frazionaria viene **troncata**

Nello standard di C++ 11 è stata introdotta l’inizializzazione con `{}`, che impedisce la narrowing conversion generando un errore in fase di compilazione.

Esempio 1.4 (Utilizzo dell’inizializzazione che impedisce il narrowing).

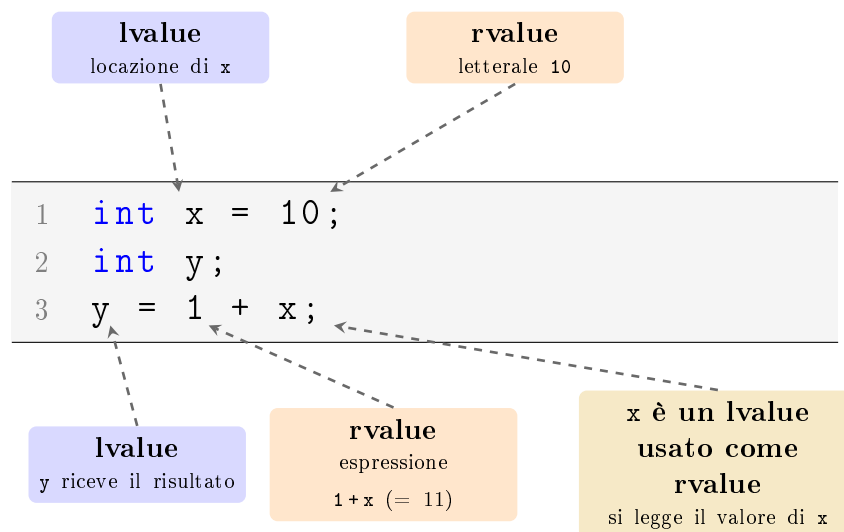
```
1 double x = 3.7;
2 int a = x;    // narrowing implicito: concesso
3 int b{x};    // narrowing vietato: errore di compilazione
```

- La riga con `=` permette la conversione da double a int.
- La riga con `{}`, secondo lo standard C++ 11, deve rifiutare il narrowing; quindi, il compilatore restituisce un errore.

1.3 Left value Right value

Definizione 1.3: Left value e Right value

- **Left value:** è un'espressione che identifica una locazione di memoria persistente, cioè un oggetto che ha un indirizzo a cui si può accedere anche dopo la valutazione dell'espressione. Il nome "left value" deriva dal fatto che storicamente è ciò che può stare a sinistra di un operatore di assegnamento.
- **Right value:** è un'espressione temporanea senza un indirizzo persistente, che può stare solo a destra.

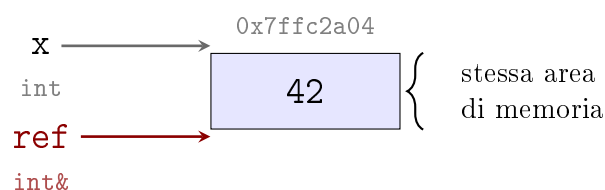


2 Passaggio dei parametri

Prima di addentrarci su come funzionano i passaggi per parametro in C++, andiamo a definire le **reference**.

Definizione 2.1: Reference

In C++ una reference è un alias per una variabile, ossia un nome alternativo che fa da riferimento all'area di memoria occupato dalla variabile originale. Una volta inizializzata, una reference non può essere riassegnata per riferirsi ad un altro oggetto.



In C++ il passaggio dei parametri può essere fatto per valore e per riferimento.

Esempio 1.5 (Swap passaggio per valore).

Nel passaggio per valore, alla funzione viene passata una **copia** del valore dell'argomento effettivo. Le modifiche ai parametri formali all'interno della funzione non influenzano le variabili originali nel chiamante.

```
1 void swap(int a, int b){
2     int c = a;
3     a = b;
4     b = c;
5     // Qui a e b sono effettivamente scambiati,
6     // ma sono solo copie locali!
7 }
8
9 int main() {
10     int x = 5;
11     int y = 3;
12
13     swap(x, y); // Passiamo i valori 5 e 3
14
15     cout << x << " " << y;
16     return 0;
17 }
```

L'output del programma sarà `5 3`: `x` e `y` non vengono scambiati perché `a` e `b` sono solo copie locali che vengono distrutte alla fine della funzione.

Esempio 1.6 (Passaggio per copia di una struct).

Consideriamo una struttura `st` definita come segue:

```
1 struct st {
2     int a;
3     int* b;
4     double c;
5 };
```

Quando si passa una **struct** per valore, il parametro formale **par** riceve una **copia bit per bit** di tutti i campi del parametro effettivo. Quanti byte vengono copiati dipende dalla composizione della struttura e viene calcolato **staticamente** dal compilatore tramite `sizeof()`.

Nel caso di **allocazione statica** del campo `b` (ad esempio `int b[20]`), `sizeof(st)` vale `21 int + 1 double` e l'intera sequenza di byte viene copiata. In questo scenario non ci sono problemi: la copia è completa e autonoma.

Il discorso cambia con l'**allocazione dinamica**. Se `b` è un puntatore a un array allocato con `malloc`, `sizeof(st)` vale `int + puntatore + double`: viene copiato solo il

valore del puntatore, non l'array a cui esso punta. Si creano così due **puntatori confluenti**: `x.b` e `par.b` puntano alla stessa area di memoria, il che genera un **problema di ownership**: chi è responsabile di liberare quella memoria? Una chiamata a `free(par.b)` all'interno di `foo` lascerebbe `x.b` come puntatore dangling.

```
1  struct st {
2      int    a;
3      int*   b;
4      double c;
5  };
6
7  void foo(st par) {
8      par.a    = 12;
9      par.b[0] = 33;    // modifica l'array originale! (
                          // puntatori confluenti)
10     par.c    = 3.14;
11 }
12
13 int main() {
14     st x;
15     x.b = (int*) malloc(sizeof(int) * 20);
16     foo(x);
17     cout << x.a;      // stampa 0: x.a non e' stato
                          // modificato
18     return 0;
19 }
```

Si distinguono quindi due tipi di copia:

- **Shallow copy** (copia superficiale): viene copiato solo il valore del puntatore; i due puntatori condividono la stessa area di memoria.
- **Deep copy** (copia profonda): viene copiato anche il contenuto dell'area puntata; le due strutture sono completamente indipendenti.

Esempio 1.7 (malloc vs new).

In C++ esistono due modi per allocare memoria dinamicamente: la funzione `malloc` (ereditata dal C) e l'operatore `new` (introdotto dal C++). La differenza fondamentale è che `malloc` è una **funzione di libreria**, mentre `new` è un **operatore del linguaggio**.

Questa distinzione ha una conseguenza pratica importante: essendo `new` un operatore, è **tipato**. `malloc` restituisce un `void*`, cioè un puntatore generico che va esplicitamente castato al tipo desiderato; `new` invece restituisce direttamente un puntatore al tipo specificato (ad esempio `int*`), senza bisogno di cast.

```

1 // malloc: restituisce void*, richiede cast esplicito
2 int* p1 = (int*) malloc(sizeof(int) * 10);
3 free(p1);
4
5 // new: restituisce int*, nessun cast necessario
6 int* p2 = new int[10];
7 delete[] p2;

```

Alla **new** corrisponde l'operatore **delete** (o **delete[]** per gli array), che libera la memoria allocata dinamicamente, analogamente a **free** per **malloc**.

A differenza di linguaggi come Java o Python, il C++ **non ha un garbage collector**. In quei linguaggi, il garbage collector è un programma che si attiva in modo trasparente, controlla tutti gli oggetti allocati in memoria e verifica se esistono ancora puntatori che vi fanno riferimento: se non ne esistono, libera automaticamente quella memoria. In C++ la gestione della memoria è invece completamente a carico del programmatore: ogni area allocata con **new** deve essere esplicitamente liberata con **delete**, pena **memory leak**.

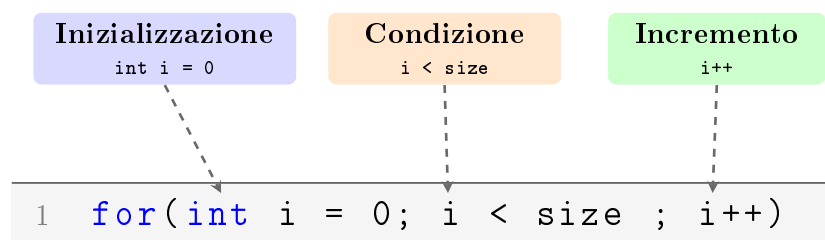
3 Uso preferenziale dei cicli

In C++(e in generale), si usano tre tipi di ciclo in base a quando conosci il numero di iterazioni e quando vuoi fare il test della condizione. Quando si sceglie tra **while**, **do...while**, **for** (e range-based **for**), la domanda vera è sempre: che tipo di ripetizione voglio esprimere?

3.1 Uso preferenziale del for

Il costrutto **for** è il più adatto quando il numero di iterazioni è collegato a un contatore che parte da un valore iniziale, viene aggiornato regolarmente e rimane all'interno di un certo intervallo.

La forma generale è:



e ha il vantaggio di riassumere in una sola riga tutte le informazioni sul ciclo: dove parte, quando si ferma, come si aggiorna il contatore.

Il range-based **for** (**for (auto x : v)**) è un caso particolare ancora più leggibile quando vogliamo dire “per ogni elemento della collezione **v**”; nasconde i dettagli sugli indici e mette l'attenzione sui dati.

In sintesi, se possiamo formulare il problema come “ripeti questa azione per tutti i valori di un contatore” o “per ogni elemento di una struttura”, il `for` (o il range-based `for`) è la scelta preferenziale.

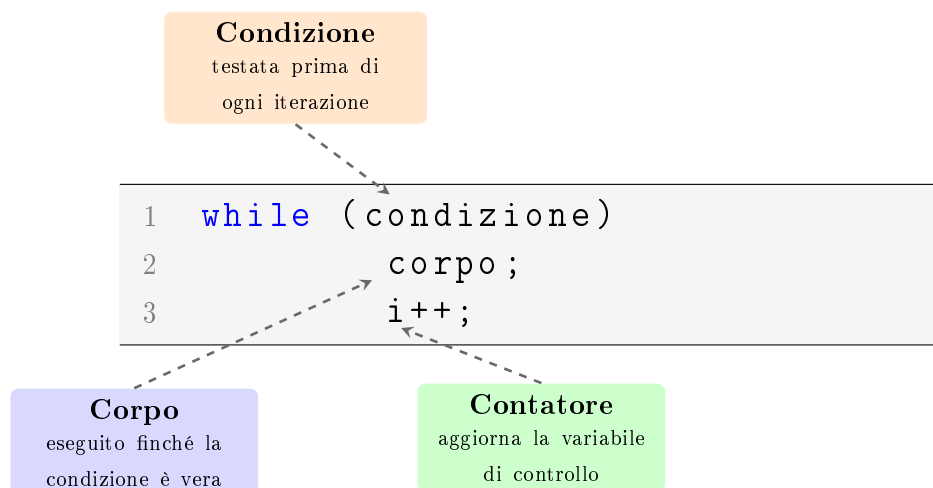
3.2 While

Il `while` è il ciclo più generale e si usa quando la durata della ripetizione dipende da una condizione logica che non necessariamente è legata a un semplice contatore.

Il ciclo `while` viene utilizzato nei seguenti casi:

- **Sentinel-controlled:** Qui il ciclo continua finché il valore letto non è uguale a un valore sentinella scelto apposta (per esempio -1), che non rappresenta un dato valido ma un segnale di “fine input”. È la situazione tipica quando non sappiamo quanti valori l’utente inserirà: leggiamo e processiamo ogni valore, e ci fermiamo quando arriva il sentinel.
- **Flag-controlled:** In un ciclo flag-controlled la condizione di continuazione dipende da una variabile booleana (il flag) che rappresenta lo stato del problema, per esempio “abbiamo già trovato quello che cercavamo?”. All’inizio il flag è impostato in modo da far entrare nel ciclo, e durante le iterazioni il programma aggiorna il flag; quando il flag assume il valore che significa “finito”, il ciclo termina. Questo schema è utile quando la fine della ripetizione dipende da un evento logico interno (trovare un elemento, soddisfare un vincolo, ecc.).
- **EOF-controlled:** Nel caso EOF-controlled, la condizione del `while` è legata alla disponibilità di nuovi dati in uno stream di input (file o tastiera). Un pattern tipico è `while (in » x)` oppure `while (cin » x)`, che fa proseguire il ciclo finché l’operazione di lettura ha successo. Questo è il modo preferenziale per elaborare tutti i dati di un file senza conoscere in anticipo quanti record contiene.

La forma generale è:

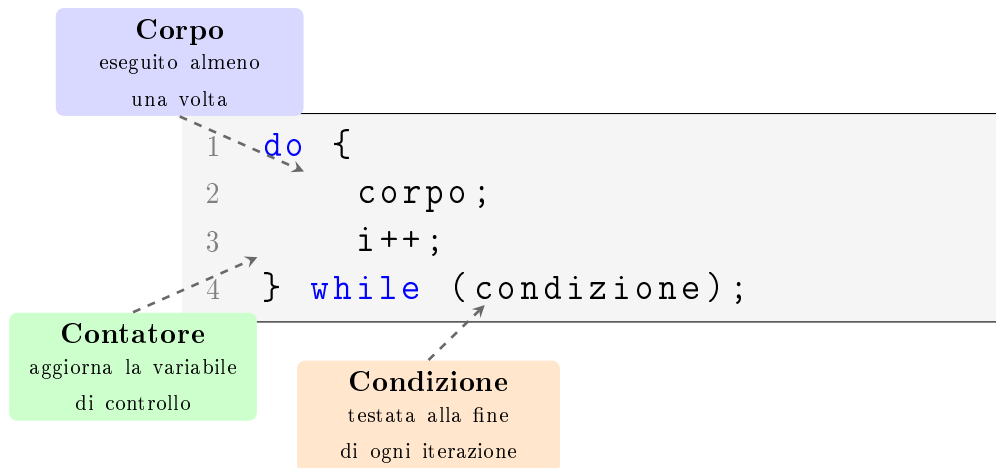


In generale il `while` è la scelta migliore quando vogliamo esprimere un processo che continua “finché una certa situazione rimane vera”, dove la situazione può dipendere da input, stato interno o dalla fine dei dati.

4 Do while

Il `do...while` si distingue perché la condizione viene controllata alla fine dell'iterazione, quindi il corpo del ciclo viene eseguito almeno una volta in ogni caso. È particolarmente adatto quando la logica naturale è “esegui questo blocco, poi decidi se ripeterlo”: è il caso tipico dei menu interattivi, dove bisogna mostrare le opzioni almeno una volta prima di chiedere se l'utente vuole continuare.

La forma generale del ciclo `do...while` è:



Situazione	Ciclo	Motivo
Iterazioni da 1 a N	<code>for</code>	Contatore naturale
Input finché sentinella	<code>while</code>	Valore sentinella per terminare
Menu ripetuto all'utente	<code>do...while</code>	Corpo eseguito almeno una volta
Scansione array con indice	<code>for</code>	Indice da 0 a size-1
Condizione logica complessa	<code>while</code>	Test separato dal setup

Un valore sentinella (sentinel) è un valore speciale non valido come dato normale, usato solo per indicare la fine dell'input o del ciclo.

5 Programmazione strutturata

In questo corso e in “generale” è preferibile utilizzare il paradigma della **programmazione strutturata**, questo paradigma impone di scrivere i programmi usando re costrutti di controllo, senza ricorrere a salti incondizionati come `goto`, `break`, `continue`.

I tre costrutti fondamentali:

- **Sequenza:** Le istruzioni vengono eseguite una dopo l'altra, nell'ordine in cui appaiono nel codice, Non c'è salto, non c'è scelta: si esegue A, poi B, poi C.
- **Selezione:** In base a una condizione booleana si sceglie quale ramo eseguire. In C++ sono l' `if`, `if...else` e lo `switch`. Solo uno dei rami viene eseguito, poi il controllo riprende dopo la struttura.

- **Ripetizione:** Un blocco di istruzioni viene eseguito ripetutamente finchè una condizione è vera. In C++ sono il `while`, il `do...while` e il `for`.

Teorema 5.1: Böhm–Jacopini (1966)

Ogni algoritmo calcolabile può essere espresso usando soltanto i tre costrutti di sequenza, selezione e iterazione, senza mai ricorrere a salti incondizionati.

CAPITOLO 2

VETTORI E STRINGHE

1	Array	20
1.1	Definizione e idea intuitiva di array	20
1.2	Limiti degli array e motivazione per strutture più astratte	20
2	Vector	21
2.1	Idea generale	21
2.2	Struttura interna	21
2.3	Metodi principali	22
2.4	Passaggio dei parametri	22
3	Bubble sort	23
3.1	Il codice completo	27
4	String	27
4.1	Differenze rispetto a <code>char*</code>	27
4.2	Input e output	27
4.3	Operazioni fondamentali	27
4.4	Concatenazione e costruzione incrementale	28
4.5	Estrazione, ricerca e sostituzione	29
5	List	30
5.1	Dichiarazione e dimensione	30
5.2	Inserimento e rimozione in testa e in coda	30
6	Problemi e pattern di programmazione	31
6.1	Pattern esistenziale: é presente?	31
6.2	Pattern universale: tutti soddisfano la proprietà?	33
6.3	Esempio di esecuzione	35
6.4	Ottimizzazione: terminazione anticipata	37
6.5	Esempio di early termination	38

1 Array

1.1 Definizione e idea intuitiva di array

Definizione 1.1: Array

Un array è una collezione di un numero fisso di elementi, tutti dello stesso tipo, memorizzati in celle di memoria contigue.

Dal punto di vista sintattico, la dichiarazione di un array in C++ segue la forma generale:

```
1      dataType arrayName[intExp];
```

dove `dataType` indica il tipo degli elementi, `arrayName` è il nome attribuito all'array e `intExp` è un'espressione costante positiva che specifica il numero di componenti. È importante osservare che la dimensione così indicata rimane fissa per tutta la durata di vita dell'array: una volta dichiarato, non è possibile aumentare o diminuire il numero di elementi senza creare una nuova struttura e copiarvi i dati.

1.2 Limiti degli array e motivazione per strutture più astratte

Gli array forniscono un meccanismo essenziale per rappresentare sequenze di valori omogenei in memoria contigua, con accesso diretto tramite indice. Tuttavia, proprio le caratteristiche che li rendono semplici ed efficienti ne evidenziano anche alcuni limiti strutturali, che diventano rilevanti nella progettazione di programmi complessi.

Un primo limite riguarda la rigidità della dimensione. La lunghezza di un array deve essere nota al momento della dichiarazione e rimane invariata per tutta la vita dell'array.

Ciò obbliga il programmatore a stimare in anticipo il numero massimo di elementi necessari: una scelta troppo prudente porta a spreco di memoria, mentre una stima troppo ottimistica può risultare insufficiente in casi reali d'uso. In altre parole, l'array è un contenitore statico, poco adatto a modellare collezioni la cui cardinalità può variare in modo significativo durante l'esecuzione.

Un secondo limite è legato alla mancanza di informazione incorporata sulla dimensione logica. Quando un array viene passato a una funzione, esso “decade” in un puntatore al suo primo elemento; la funzione riceve quindi solo un indirizzo di memoria, e non ha più accesso diretto alla dimensione dell'array originario. Di conseguenza, è necessario trasmettere esplicitamente un parametro aggiuntivo che rappresenti la lunghezza effettiva della sequenza, con il rischio di introdurre incoerenze tra i dati e le informazioni di supporto, oppure di incorrere in accessi fuori dai limiti qualora tali parametri non vengano gestiti con attenzione.

Inoltre, gli array non mettono a disposizione operazioni ad alto livello per la gestione dinamica dei dati. Non esistono primitive del linguaggio per inserire o rimuovere elementi all'interno della sequenza, per ridimensionare il contenitore o per verificarne automaticamente i limiti; tali operazioni devono essere simulate manualmente, spostando gli elementi e gestendo direttamente la memoria sottostante. Questa mancanza di astrazione aumenta

la complessità del codice applicativo e rende più probabile l'introduzione di errori legati alla gestione della memoria, agli indici e alla coerenza dei dati.

Inoltre, gli array non mettono a disposizione operazioni ad alto livello per la gestione dinamica dei dati. Non esistono primitive del linguaggio per inserire o rimuovere elementi all'interno della sequenza, per ridimensionare il contenitore o per verificarne automaticamente i limiti; tali operazioni devono essere simulate manualmente, spostando gli elementi e gestendo direttamente la memoria sottostante. Questa mancanza di astrazione aumenta la complessità del codice applicativo e rende più probabile l'introduzione di errori legati alla gestione della memoria, agli indici e alla coerenza dei dati.

2 Vector

2.1 Idea generale

Definizione 2.1: Vector

Un vector è un array dinamico: una sequenza ordinata di elementi omogenei memorizzati in celle di memoria contigue, la cui dimensione può variare durante l'esecuzione del programma. A differenza degli array statici, il `std::vector` integra la gestione automatica della memoria e mette a disposizione operazioni ad alto livello per inserire, rimuovere e accedere agli elementi.

Dal punto di vista pratico, per utilizzare un `std::vector` in C++ la prima operazione necessaria è includere l'header della libreria standard che ne definisce l'interfaccia tramite la direttiva `#include <vector>`.

Una dichiarazione tipica è `std::vector<int> v;`, che crea un vettore di interi vuoto, la cui dimensione iniziale è 0. In alternativa, la dichiarazione `std::vector<int> v(100);` costruisce un vettore con 100 elementi già presenti, tutti inizializzati a 0.

Il punto chiave è che la dimensione logica del container (non il tipo `std::vector<int>` in sé) non è più una costante fissata staticamente come negli array, ma uno stato interno dell'oggetto che può cambiare durante l'esecuzione del programma.

2.2 Struttura interna

Dal punto di vista implementativo, `std::vector<T>` è un oggetto che incapsula tre campi fondamentali:

- un **puntatore** al buffer contiguo in memoria, dove sono effettivamente memorizzati gli elementi di tipo `T`;
- **size**: un intero che rappresenta il numero di elementi attualmente presenti nel vettore;
- **capacity**: un intero che indica quanta memoria è stata riservata, ovvero quanti elementi possono essere contenuti prima che sia necessaria una riallocazione.

In generale si ha $\text{size} \leq \text{capacity}$: quando si inserisce un elemento con `push_back` e la `size` raggiunge la `capacity`, il vector alloca automaticamente un nuovo buffer più grande (tipicamente raddoppia la `capacity`), copia gli elementi

2.3 Metodi principali

Su questo oggetto è possibile chiamare i seguenti metodi:

- `v.size()`: restituisce il numero di elementi logici attualmente contenuti nel vettore.
- `v.at(i)`: restituisce il riferimento all'elemento in posizione `i`, controllandone i limiti. Se `i` è fuori dall'intervallo valido, lancia un'eccezione di tipo `std::out_of_range`, causando un errore controllato anziché un comportamento indefinito come con l'operatore `[]`.
- `v.resize(n)`: modifica la dimensione logica del vettore portandola a `n`. Se `n` è maggiore della size attuale, i nuovi elementi vengono inizializzati al valore di default del tipo (ad esempio `0` per `int`); se `n` è minore, gli elementi in eccesso vengono rimossi.
- `v.push_back(x)`: aggiunge un nuovo elemento in coda al vettore, incrementando la size di 1. Se necessario, viene eseguita una riallocazione automatica del buffer interno.

2.4 Passaggio dei parametri

Esempio 2.1 (Passaggio dei parametri).

Un `std::vector` viene tipicamente passato alle funzioni **per riferimento**, evitando la copia dell'intero buffer e permettendo alla funzione di modificare il contenuto originale.

```

1  void foo(vector<int>& v) {
2      v.at(2) = 0;
3  }
4
5  int main() {
6      vector<int> v {1, 2, 3, 4, 5};
7      foo(v);
8      cout << v.at(2);
9      return 0;
10 }
```

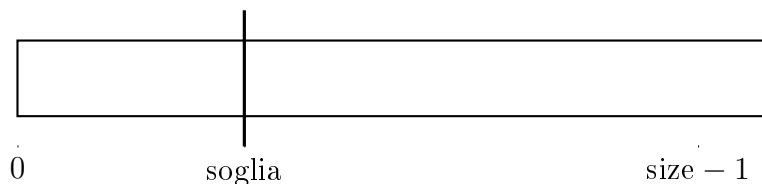
In questo esempio, `foo` riceve il vettore per riferimento e modifica l'elemento in posizione 2. La chiamata `cout<< v.at(2)` nel `main` stamperà quindi `0`, confermando che la modifica ha effetto sull'oggetto originale e non su una copia.

Il simbolo `&` nella firma `vector<int>& v` implica che `v` è un **riferimento** al vettore originale: all'interno di `foo` non viene creata alcuna copia, e qualunque modifica agli elementi (ad esempio tramite `v.at(i)`) si riflette direttamente sul vettore nel `main`.

Se invece si omettesse il simbolo `&`, scrivendo `void foo(vector<int> v)`, il parametro verrebbe passato **per valore**: la funzione riceverebbe una copia locale del vettore, tutte le modifiche rimarrebbero confinate a quella copia e nessun effetto si propagherebbe al vettore originale nel chiamante.

3 Bubble sort

Sia v un vettore con una soglia arbitraria. Immaginiamo che la porzione del vettore che va da 0 a soglia sia ordinata, mentre la porzione che va da soglia a $size - 1$ sia disordinata.

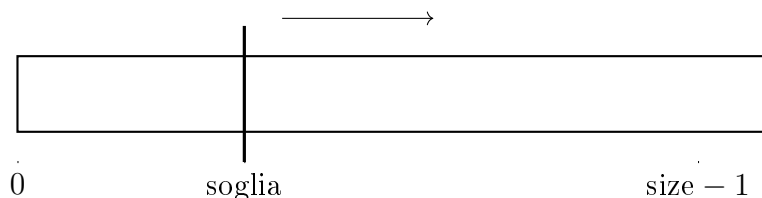


Per ordinata intendiamo che gli elementi della porzione $[0, soglia]$ siano in ordine crescente e siano tutti più piccoli degli elementi che stanno nella porzione $[soglia, size - 1]$, in cui gli elementi sono distribuiti in modo arbitrario. Inoltre, gli elementi in questa seconda porzione sono tutti maggiori o uguali agli elementi che stanno in $[0, soglia - 1]$.

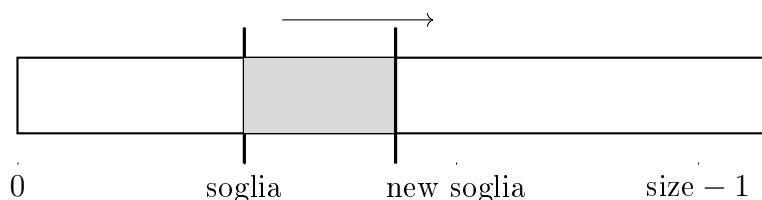
Potenzialmente ci troveremo nel caso in cui il vettore sia disordinato e quindi avremo la soglia uguale a zero. Questo ci fa leggere la condizione come la porzione che va da 0 a -1 , ovvero la porzione vuota, che consideriamo ordinata. La porzione $[0, size - 1]$ è quindi disordinata, cioè l'intero vettore.

Pertanto, quando analizziamo il vettore per la prima volta, ci troviamo in questa situazione nella maggior parte dei casi. Quando invece il vettore è completamente ordinato, la soglia varrà $size$, poiché la parte tra 0 e $size - 1$ sarà ordinata, e quella tra $size$ e $size - 1$ sarà l'intervallo vuoto (visto che l'estremo superiore è minore dell'estremo inferiore).

Ad ogni iterazione del ciclo dovrò spingere la soglia verso destra.



Devo prendere l'elemento più piccolo che si trova a destra; devo portarlo nella posizione successiva alla soglia e, a questo punto, la soglia viene spostata a destra, poiché ho rispettato la regola: ho pescato sicuramente l'elemento che è maggiore o uguale a quelli nella porzione $[0, soglia - 1]$ e l'ho posizionato subito dopo.



Esempio 2.2 (Struttura base del bubble sort).

L'ordinamento viene implementato scorrendo il vettore dal fondo verso la soglia, confrontando gli elementi a due a due e scambiandoli se necessario.

```

1 void bubble(vector<int> &v){
2     for (int soglia = 0; soglia < v.size(); soglia++) {
3         // La soglia deve prendere l'elemento
4         // minore del range [soglia, size-1]
5         // e metterlo in posizione soglia
6     }
7 }

```

L'ordinamento avviene se prendo il vettore e lo scorro nella direzione opposta rispetto a come scorre la soglia: dal fondo verso l'inizio. Posso considerare gli elementi a due a due: il penultimo con l'ultimo, e se l'ultimo è più piccolo, li scambio. Altrimenti, se sono già ordinati, non faccio nulla. Poi confronto il terzultimo con il penultimo, e così via.

Cosa succede quando arrivo all'elemento più piccolo del range $soglia-size-1$? L'elemento più piccolo verrà progressivamente spostato verso sinistra.

Esempio 2.3 (Passata del bubble sort su un vettore). *Immaginiamo che la porzione che stiamo analizzando sia questa:*

7	2	5	0	1
---	---	---	---	---

Comincio ad analizzare l'1 con lo 0:

- *Confronto l'1 con lo 0: sono disordinati.*

7	2	5	0	1
---	---	---	---	---

- *Confronto lo 0 con il 5: scambio.*
- *Confronto lo 0 con il 2: scambio.*
- *Confronto lo 0 con il 7: scambio.*

Il valore 0 viene portato in prima posizione, e ottengo il seguente vettore:

0	7	2	5	1
---	---	---	---	---

Esempio 2.4 (Bubble sort completo).

*Implementazione completa con la funzione **swap** e il doppio ciclo.*


```

1 void swap(int &a, int &b){
2     int c = a;
3     a = b;
4     b = c;
5 }
6
7 void bubble(vector<int> &v){
8     for (int soglia = 0; soglia < v.size() - 1; soglia++)
9     {
10         for (int i = v.size() - 1; i > soglia; i--) {
11             if (v.at(i) < v.at(i - 1))
12                 swap(v.at(i), v.at(i - 1));
13         }
14     }
15 }

```

- **Dove devono fermarsi questi indici?**

Partendo da `size - 1`, il primo confronto è tra `v[size-1]` e `v[size-2]`, che è corretto. Se mi fermassi a `soglia`, l'ultimo confronto sarebbe tra `soglia` e `soglia-1`. Tuttavia, l'invariante del ciclo ci garantisce che l'intervallo $[0, soglia - 1]$ è ordinato. Quindi non serve confrontare gli elementi in questa parte. Concludiamo quindi che il ciclo può fermarsi a `i > soglia`.

- **È corretto che i parta da `size - 1`?**

Sì, perché se la `soglia` vale `size - 1`, il ciclo interno non si esegue (nessun confronto), ed è corretto: un vettore con un solo elemento è già ordinato. Inoltre, se il vettore ha un solo elemento, l'intero ciclo esterno farà solo una iterazione inutile, che possiamo evitare usando la condizione `soglia < v.size() - 1`.

Mettere `size - 1` come condizione di arresto implica, secondo l'invariante del ciclo, che se la parte ordinata va da `0` a `size - 2` e la parte disordinata contiene un unico elemento, allora tutto l'array può essere considerato ordinato.

Attenzione a `size_t`

`v.size()` restituisce un valore di tipo `size_t`, che è *unsigned*. Se il vettore è vuoto, `v.size() - 1` non vale `-1`, ma causa un *underflow* aritmetico, producendo un valore enorme ($2^{64} - 1$ su architetture a 64 bit). Il confronto `soglia < v.size() - 1` risulterebbe quindi sempre vero, portando a un comportamento indefinito. Per evitare il problema, è sufficiente aggiungere un controllo iniziale `if (v.empty()) return;` oppure effettuare un cast esplicito `soglia < (int)v.size() - 1`.

Ottimizzazione: terminazione anticipata

La versione base del bubble sort esegue sempre tutti i passaggi, anche se il vettore diventa ordinato prima di completarli. Possiamo ottimizzare l'algoritmo introducendo una varia-

bile booleana `check` che tenga traccia di un fatto semplice: *durante l'ultimo passaggio, ho effettuato almeno uno scambio?*

L'idea è la seguente:

- All'inizio di ogni passata, assumo che il segmento sia già ordinato (`check = false`, cioè “nessuno scambio effettuato”).
- Se durante la passata effettuo almeno uno scambio, imposto `check = true`.
- Al termine della passata, se `check` è rimasto `false`, significa che non ho fatto alcuno scambio: il vettore è ordinato e posso terminare.

Attenzione: è fondamentale che l'assegnamento `check = true` si trovi **dentro** il blocco `if`. Se lo si mette fuori dall'`if`, la variabile viene modificata a ogni iterazione indipendentemente dallo scambio, e l'ottimizzazione non funziona.

Poiché la variabile `check` viene modificata nel corpo del ciclo e usata nella condizione di arresto, la scelta stilisticamente più corretta è usare un ciclo `while` per il ciclo esterno:

Esempio 2.5 (Bubble sort ottimizzato con terminazione anticipata).

Uso di un ciclo `while` e della variabile `check` per terminare anticipatamente quando il vettore è già ordinato.

```

1  void bubble(vector<int> &v) {
2      if (v.empty()) return;
3
4      bool check = true;
5      int soglia = 0;
6
7      while (soglia < (int)v.size() - 1 && check) {
8          check = false;
9          for (int i = v.size() - 1; i > soglia; i--) {
10             if (v[i] < v[i - 1]) {
11                 swap(v[i], v[i - 1]);
12                 check = true; // scambio avvenuto!
13             }
14         }
15         soglia++;
16     }
17 }
```

Stabilità

Il bubble sort è un algoritmo **stabile**: se due elementi hanno lo stesso valore, il loro ordine relativo nel vettore originale viene preservato. Questo accade perché lo scambio avviene solo quando $v[i] < v[i-1]$ (con disuguaglianza stretta): due elementi uguali non vengono mai scambiati tra loro.

Ordinamento in-place

Il bubble sort ordina il vettore **in-place**, cioè senza richiedere memoria aggiuntiva proporzionale alla dimensione dell'input. L'unica memoria extra utilizzata è quella per le variabili ausiliarie (`soglia`, `i`, `check` e la variabile temporanea dello `swap`), quindi lo spazio aggiuntivo è $O(1)$.

3.1 Il codice completo

4 String

`std::string` è una classe della libreria standard che incapsula una sequenza mutabile di caratteri, con molte istruzioni e operazioni disponibili. Dal punto di vista strutturale è molto simile a un `vector<char>`.

4.1 Differenze rispetto a `char*`

A differenza delle C-string (`char*`), `std::string` offre funzionalità più ricche e sicure:

- **Gestione automatica della memoria:** la stringa si ridimensiona automaticamente quando si aggiungono o rimuovono caratteri, senza bisogno di allocare o deallocare memoria manualmente.
- **Lunghezza in tempo costante:** `s.size()` restituisce il numero di caratteri in tempo $O(1)$, poiché la lunghezza è mantenuta come campo interno dell'oggetto.

4.2 Input e output

Per leggere e scrivere stringhe si utilizzano i flussi standard: `cin` per l'input e `cout` per l'output.

4.3 Operazioni fondamentali

Creazione

- `string s;` — crea una stringa vuota.
- `string s = "ciao";` — inizializza la stringa con il letterale `"ciao"`.

Assegnamento

- `s = "hello";` — sostituisce l'intero contenuto di `s` con `"hello"`.
- `s = s1;` — copia il contenuto di `s1` in `s`.

Lunghezza

`s.size()` e `s.length()` sono equivalenti e restituiscono il numero di caratteri della stringa in tempo $O(1)$.

Accesso ai caratteri

Come per il vector, è possibile accedere ai singoli caratteri tramite `s.at(i)`, che effettua il controllo dei limiti, oppure tramite l'operatore `s[i]`. Ad esempio: `s.at(i) = 'z'`; modifica il carattere in posizione `i`.

Un pattern comune è iterare sui caratteri con un range-for e modificarli in place. L'esempio seguente converte tutti i caratteri minuscoli in maiuscoli:

Esempio 2.6 (Conversione minuscolo → maiuscolo).

```

1  for (char& c : s) {
2      if (c >= 'a' && c <= 'z')
3          c = c - 'a' + 'A';
4  }
5

```

Confronto e ordinamento

Le stringhe supportano gli operatori di confronto standard:

- `s1 == s2` — vero se le due stringhe hanno la stessa lunghezza e gli stessi caratteri in ogni posizione.
- `s1 != s2` — vero se almeno un carattere è diverso o le lunghezze sono diverse.
- `s1 < s2` — confronto lessicografico: restituisce vero se `s1` precede `s2` nell'ordine del dizionario.

4.4 Concatenazione e costruzione incrementale

Per costruire una nuova stringa per concatenazione si usa l'operatore `+`:

- `string s2 = s1 + s2;` — crea una nuova stringa concatenando `s1` e `s2`.

Per la **concatenazione in-place** (senza creare oggetti temporanei) si usa l'operatore `+=`:

- `s1 += s2;` — appende `s2` in fondo a `s1`, modificandola direttamente.
- `s1 += '!';` — aggiunge un singolo carattere in coda.

Per la **costruzione iterativa** carattere per carattere o blocco per blocco:

- `s.push_back('o');` — aggiunge il carattere `'o'` in fondo alla stringa.
- `s.append("ciao");` — appende la sottostringa `"ciao"` in fondo.

4.5 Estrazione, ricerca e sostituzione

Operazioni di slicing con `substr`

- `s.substr(pos)` — restituisce la sottostringa che va dalla posizione `pos` fino alla fine della stringa.
- `s.substr(pos, len)` — restituisce la sottostringa di lunghezza `len` a partire dalla posizione `pos`.

```
1  string s = "ciao mondo";
2  string t = s.substr(5);      // "mondo"
3  string u = s.substr(0, 4);   // "ciao"
4
```

Operazioni di ricerca con `find`

- `s.find("ciao")` — restituisce l'indice della prima occorrenza della sottostringa "ciao" in `s`, oppure `string::npos` se non trovata.
- `s.find('a')` — versione con singolo carattere.

L'esempio seguente estrae la seconda parola di una stringa cercando il primo spazio:

Esempio 2.8 (Prima parola dopo uno spazio).

```
1  size_t pos = s.find(' ');
2  if (pos != string::npos)
3      string seconda = s.substr(pos + 1);
4
```

Sostituzione con `replace`

`s.replace(pos, len, "nuovo_testo")` sostituisce `len` caratteri a partire dalla posizione `pos` con la stringa "nuovo_testo". In un'unica operazione cancella una porzione della stringa e inserisce il testo di rimpiazzo, che può avere lunghezza diversa.

Inserimento con `insert`

`s.insert(pos, "xxx")` inserisce la stringa "xxx" alla posizione `pos`, spostando in avanti i caratteri successivi.

Cancellazione con erase

`s.erase(pos, len)` cancella `len` caratteri a partire dalla posizione `pos`.

L'esempio seguente mostra l'uso combinato di `insert` e `erase` su una stringa:

Esempio 2.9 (Inserimento e cancellazione).

```
1  string s = "cane";
2  s.insert(1, "ial"); // s diventa "cialane"
3  s.erase(4, 3);      // s diventa "cial"
4
```

5 List

`std::list<T>` è una **lista doppiamente concatenata**: ogni nodo contiene un elemento e due puntatori, rispettivamente al nodo precedente e al nodo successivo. A differenza del `vector`, gli elementi non sono contigui in memoria ma allocati separatamente nello heap e collegati tra loro tramite i puntatori.

5.1 Dichiarazione e dimensione

- `list<int> l;` — crea una lista vuota di interi.
- `list<int> l(n);` — crea una lista con `n` elementi inizializzati al valore di default (0 per `int`).
- `l.size();` — restituisce il numero di elementi presenti nella lista.

5.2 Inserimento e rimozione in testa e in coda

La `list` supporta inserimento e rimozione in tempo $O(1)$ sia in testa che in coda:

- `l.push_back(x);` — aggiunge l'elemento `x` in fondo alla lista.
- `l.push_front(x);` — aggiunge l'elemento `x` in testa alla lista.
- `l.pop_back();` — rimuove l'ultimo elemento.
- `l.pop_front();` — rimuove il primo elemento.

6 Problemi e pattern di programmazione

Problema 2.1

Funzione di lettura di un vettore.

Scrivere una funzione `leggi` che riceva un vettore di interi per riferimento, legga da standard input il numero di elementi e li carichi uno ad uno nel vettore.

Metodologia risolutiva proposta:

Leggere prima la dimensione n , ridimensionare il vettore con `v.resize(n)`, poi caricare i valori uno ad uno sfruttando il range-for con riferimento.

Soluzione

Un pattern comune è definire una funzione `leggi` che riceve un vettore per riferimento, legge da standard input il numero di elementi e poi li carica uno ad uno nel vettore:

1. Legge un intero n dallo standard input, che rappresenta il numero di elementi del vettore.
2. Ridimensiona il vettore con `v.resize(n)`.
3. Legge gli n valori uno ad uno, caricandoli direttamente negli elementi del vettore.

```
1 void leggi(std::vector<int>& v) {  
2     int n;  
3     cin >> n;  
4     v.resize(n);  
5     for (int& x : v)  
6         cin >> x;  
7 }  
8
```

Il ciclo range-for con `int& x` dichiara `x` come **reference** all'elemento corrente del vettore. A ogni iterazione `x` si lega successivamente a ciascun elemento di `v`: poichè è una reference, l'istruzione `cin >> x` scrive direttamente nell'elemento del vettore, modificandolo in place.

6.1 Pattern esistenziale: é presente?

Un pattern ricorrente sui vettori è verificare se esiste almeno un elemento che soddisfa una certa proprietà — si parla di **proprietà esistenziale**. L'idea è usare un flag booleano

`found` inizializzato a `false`, che viene impostato a `true` non appena si trova un elemento che soddisfa la condizione; il ciclo termina anticipatamente grazie alla condizione `&& !found`.

Problema 2.2

Presenza di un elemento.

Data una funzione `presente` che riceva un vettore di interi `const vector<int>& v` e un intero `x`, decidere se `x` è presente nel vettore.

Metodologia risolutiva proposta:

Usare un flag `found = false` e scorrere il vettore fino a trovare l'elemento oppure fino alla fine; restituire il flag.

Soluzione

```
1  bool presente(const vector<int>& v, int x) {
2      bool found = false;
3      int i = 0;
4      while (i < v.size() && !found) {
5          if (v.at(i) == x)
6              found = true;
7          i++;
8      }
9      return found;
10 }
11
```


6.2 Pattern universale: tutti soddisfano la proprietà?

Il duale della proprietà esistenziale è la **proprietà universale**: vogliamo verificare se *tutti* gli elementi soddisfano una condizione. In questo caso il flag parte a `true` e viene abbattuto non appena si trova un elemento che *non* soddisfa la condizione.

Problema 2.3

Tutti i pari.

Data la funzione `tuttiPari` che riceva un vettore di interi, decidere se tutti gli elementi sono pari.

Metodologia risolutiva proposta:

Usare un flag `found = true` e scorrere il vettore; se si incontra un elemento dispari abbattere il flag e interrompere il ciclo.

Soluzione

```
1  bool tuttiPari(const vector<int>& v) {
2      bool found = true;
3      int i = 0;
4      while (i < v.size() && found) {
5          if (v.at(i) % 2 != 0)
6              found = false;
7          i++;
8      }
9      return found;
10 }
11
```

Problema 2.4

Implementazione del Bubble Sort.

Scrivere una funzione che ordini un vettore di interi in ordine crescente usando l'algoritmo bubble sort.

Metodologia risolutiva proposta:

Dividere il vettore in due zone: da 0 a `soglia - 1` ordinato, da `soglia` a fine disordinato. Ad ogni passata, l'elemento più piccolo della zona disordinata "galleggia" verso la posizione `soglia` attraverso scambi ripetuti, poi `soglia` avanza.

Soluzione

Il bubble sort funziona facendo “galleggiare” gli elementi nella posizione corretta attraverso scambi ripetuti tra elementi adiacenti. Abbiamo un vettore diviso in due zone da una variabile `soglia`: da indice 0 a `soglia - 1` la zona è ordinata, da `soglia` a fine vettore la zona è disordinata. All’inizio `soglia = 0` (niente è ordinato). Ad ogni passata, l’elemento più piccolo della zona disordinata “galleggia” fino alla posizione `soglia`, e poi `soglia` avanza di 1.

```

1  void bubble(vector<int>& v) {
2      if (v.empty()) return;           // (1)
3      int soglia = 0;                   // (2)
4      while (soglia < (int)v.size() - 1) { // (3)
5          for (int i = v.size() - 1; i > soglia; i--) { //
(4)
6              if (v.at(i) < v.at(i - 1)) { // (5)
7                  swap(v.at(i), v.at(i - 1)); // (6)
8              }
9          }
10         soglia++;                       // (7)
11     }
12 }
13

```

Analisi riga per riga:

1. `if (v.empty()) return;` — Se il vettore è vuoto non c’è nulla da ordinare. Senza questo controllo, `v.size() - 1` su un vettore vuoto darebbe un numero enorme (perché `size()` restituisce un `unsigned`, e `0 - 1` in `unsigned` va in overflow).
2. `int soglia = 0;` — Inizializziamo la soglia a 0: “nessun elemento è ancora nella posizione corretta”. Tutta la zona da 0 a `v.size()-1` è disordinata.
3. `while (soglia < (int)v.size() - 1)` — Il ciclo esterno continua finché la soglia non ha raggiunto la penultima posizione. Perché penultima e non ultima? Perché se tutti gli elementi tranne l’ultimo sono al posto giusto, anche l’ultimo per forza lo è. Il cast `(int)` serve perché `v.size()` è `unsigned`.
4. `for (int i = v.size() - 1; i > soglia; i-)` — Questo è il ciclo interno. Parte dall’ultimo elemento del vettore e scende verso la soglia. Perché dal fondo? Perché vogliamo far “salire” l’elemento più piccolo verso sinistra, verso la posizione soglia. `i > soglia` e non `i >= soglia` perché nel corpo del `for` confrontiamo `v[i]` con `v[i-1]`. Se `i` fosse uguale a `soglia`, allora `i-1` sarebbe `soglia - 1`, cioè andremmo a toccare la zona già ordinata, che non va toccata.

5. `if (v.at(i) < v.at(i - 1))` — Confrontiamo l'elemento corrente con quello che lo precede. Se quello corrente è più piccolo, sono nell'ordine sbagliato (perché vogliamo ordine crescente).
6. `swap(v.at(i), v.at(i - 1));` — Li scambiamo. Dopo lo scambio, l'elemento più piccolo si è spostato di una posizione verso sinistra. Questo succede ripetutamente lungo tutto il ciclo `for`, quindi l'elemento più piccolo della zona disordinata “galleggia” fino a `soglia`.
7. `soglia++;` — La passata è finita. L'elemento più piccolo della zona disordinata è arrivato in posizione `soglia`. Ora quella posizione fa parte della zona ordinata, quindi avanziamo la soglia di 1.

6.3 Esempio di esecuzione

Legenda

Ordinato (sotto soglia)
 Confronto
 Scambio
 Disordinato

Vettore iniziale: $[5, 3, 1, 4, 2]$, $soglia = 0$

Passata 1

$soglia = 0$, il `for` va da $i = 4$ fino a $i > 0$ (4 confronti)

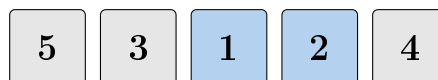
Stato iniziale:



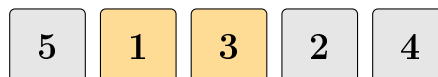
$i = 4$: $v[4] = 2 < v[3] = 4 \checkmark \Rightarrow \text{swap!}$



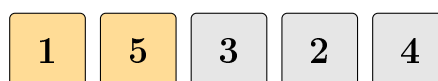
$i = 3$: $v[3] = 2 < v[2] = 1$? No $\Rightarrow$ nessuno scambio



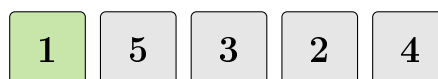
$i = 2$: $v[2] = 1 < v[1] = 3 \checkmark \Rightarrow \text{swap!}$



$i = 1$: $v[1] = 1 < v[0] = 5 \checkmark \Rightarrow \text{swap!}$



$soglia++ \Rightarrow soglia = 1$



Passata 2

soglia = 1, il **for** va da $i = 4$ fino a $i > 1$ (3 confronti)

$i = 4$: $v[4] = 4 < v[3] = 2$? No $\Rightarrow$ nessuno scambio



$i = 3$: $v[3] = 2 < v[2] = 3$ ✓ $\Rightarrow$ **swap!**



$i = 2$: $v[2] = 2 < v[1] = 5$ ✓ $\Rightarrow$ **swap!**



soglia++ $\Rightarrow$ soglia = 2



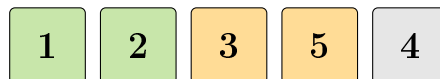
Passata 3

soglia = 2, il **for** va da $i = 4$ fino a $i > 2$ (2 confronti)

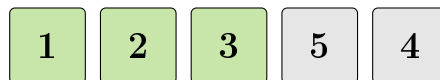
$i = 4$: $v[4] = 4 < v[3] = 3$? No $\Rightarrow$ nessuno scambio



$i = 3$: $v[3] = 3 < v[2] = 5$ ✓ $\Rightarrow$ **swap!**



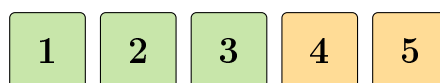
soglia++ $\Rightarrow$ soglia = 3



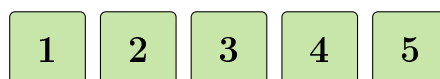
Passata 4

soglia = 3, il **for** va da $i = 4$ fino a $i > 3$ (1 confronto)

$i = 4$: $v[4] = 4 < v[3] = 5$ ✓ $\Rightarrow$ **swap!**



soglia++ $\Rightarrow$ soglia = 4



Terminazione

soglia = $4 < v.size() - 1 = 4$? **No** $\Rightarrow$ il **while** termina.

Risultato finale: $[1, 2, 3, 4, 5]$ ✓

6.4 Ottimizzazione: terminazione anticipata

Problema 2.5

Bubble Sort con early termination.

Modificare la funzione bubble sort per terminare anticipatamente se il vettore è già ordinato.

Metodologia risolutiva proposta:

Usare un flag booleano **check** che viene resettato a **false** all'inizio di ogni passata. Se durante la passata avviene almeno uno scambio, **check** viene impostato a **true**. Se alla fine di una passata **check** è ancora **false**, significa che non ci sono stati scambi e il vettore è già ordinato, quindi possiamo terminare.

Soluzione

Esempio 2.14 (Bubble sort con early termination).

```
1  void bubble(vector<int>& v) {
2      if (v.empty()) return;
3
4      bool check = true;
5      int soglia = 0;
6
7      while (soglia < (int)v.size() - 1 && check) {
8          check = false;
9          for (int i = v.size() - 1; i > soglia; i--) {
10             if (v[i] < v[i - 1]) {
11                 swap(v[i], v[i - 1]);
12                 check = true;
13             }
14         }
15         soglia++;
16     }
17 }
18
```

6.5 Esempio di early termination

Legenda

 Confronto
  Scambio
  Non toccato
  Tutto ordinato

Nota: in questo esercizio **non c'è la soglia**. Il `for` scorre sempre tutto il vettore. L'unica ottimizzazione è il flag `check`: se una passata intera non produce scambi, il vettore è già ordinato e ci fermiamo.

Vettore iniziale: $[1, 3, 2, 4, 5]$ (quasi ordinato, solo 3 e 2 invertiti)

Passata 1

`check` viene resettato a `false` $\Rightarrow$ il `for` va da $i = 4$ fino a $i > 0$ (4 confronti)

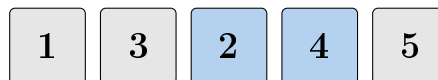
Stato iniziale:



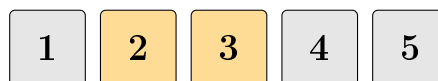
$i = 4$: $v[4] = 5 < v[3] = 4$? No $\Rightarrow$ nessuno scambio



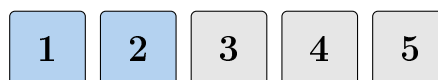
$i = 3$: $v[3] = 4 < v[2] = 2$? No $\Rightarrow$ nessuno scambio



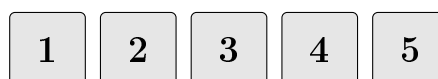
$i = 2$: $v[2] = 2 < v[1] = 3$ ✓ $\Rightarrow$ **swap!** `check = true`



$i = 1$: $v[1] = 2 < v[0] = 1$? No $\Rightarrow$ nessuno scambio



Fine passata 1: `check = true` $\Rightarrow$ è avvenuto almeno uno scambio $\Rightarrow$ **si continua**



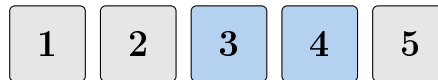
Passata 2

`check` viene resettato a `false` $\Rightarrow$ il `for` va da $i = 4$ fino a $i > 0$ (4 confronti)

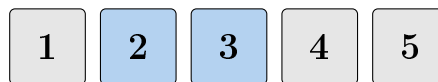
$i = 4$: $v[4] = 5 < v[3] = 4$? No $\Rightarrow$ nessuno scambio



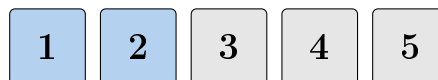
$i = 3$: $v[3] = 4 < v[2] = 3$? No $\Rightarrow$ nessuno scambio



$i = 2$: $v[2] = 3 < v[1] = 2$? No $\Rightarrow$ nessuno scambio



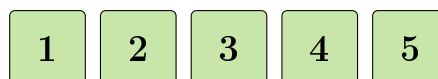
$i = 1$: $v[1] = 2 < v[0] = 1$? No $\Rightarrow$ nessuno scambio



Fine passata 2: `check = false` $\Rightarrow$ **nessuno scambio in tutta la passata!**

Questo significa che per ogni i : $v[i - 1] \leq v[i]$, cioè ogni elemento è minore o uguale del successivo.

Quindi il vettore è **già ordinato** e possiamo **terminare**.



Terminazione

Il `while` controlla `check`:

`check = false` $\Rightarrow$ il `while` termina

Risultato finale: $[1, 2, 3, 4, 5]$ ✓

CAPITOLO 3

RICORSIONE

1	Introduzione	42
1.1	Struttura formale	42
2	Ricorsione lineare	42
2.1	Post-order	43
2.2	Pre-order	44
2.3	Tail recursion	45
3	Tail recursion	47
3.1	Senza accumulatore	47
3.2	Con accumulatore	49
3.3	Accumulatore stringa	50
3.4	Accumulatori multipli paralleli	51
4	Divide et impera	52
4.1	Divide et impera con combine esplicita	52
4.2	Divide et impera senza combina, con riduzione diretta	54
5	Backtracking	57
5.1	Backtracking su scelte binarie	57
5.2	Backtracking su permutazioni	58
5.3	Backtracking con vincoli strutturali	59

1 Introduzione

Definizione 1.1: Ricorsione

La ricorsione è un metodo di risoluzione di problemi computazionali in cui la soluzione di un problema dipende dalla soluzione di istanze più piccole dello stesso problema e questo processo si realizza attraverso una funzione che richiama se stessa dall'interno del proprio corpo.

Una funzione ricorsiva è corretta e completa è sempre composta da esattamente due componenti fondamentali e inscindibili:

- Caso base: il caso più elementare del problema, per cui la risposta è nota e immediata la funzione lo risolve senza richiamare se stessa. È la condizione di arresto che garantisce la terminazione della catena ricorsiva. Senza di esso, la ricorsione continuerebbe all'infinito.
- Passo ricorsivo: il caso generale, in cui il problema non è ancora nella sua forma più semplice. La funzione lo riduce in una versione più piccola e si richiama su quella versione ridotta, avanzando progressivamente verso il caso base.

1.1 Struttura formale

Matematicamente, una definizione ricorsiva si esprime sempre in questa forma:

$$\begin{cases} \text{soluzione diretta se } n = \text{caso base} \\ g(n, f(n')) & \text{altrimenti, dove } n' < n \end{cases}$$

Dove n' è una versione ridotta di n , e g è una qualsiasi operazione che combina il risultato della chiamata ricorsiva con il valore corrente. L'importante è che ogni passo ricorsivo garantisca che n' converga verso il caso base.

Da ora consideriamo la ricorsione non come “trucco”, ma come un insieme di pattern mentali ricorrenti.

2 Ricorsione lineare

Definizione 2.1: Ricorsione lineare

Una ricorsione è **lineare** quando ogni chiamata genera esattamente una chiamata ricorsiva. La struttura è una catena: il problema di taglia n viene ridotto a un problema della stessa natura ma più piccolo, fino al caso base.

Il principio è identico all'induzione matematica: il caso base corrisponde al passo base dell'induzione, il passo ricorsivo corrisponde al passo induttivo.

Non va simulata la catena nella testa. Si assume che la funzione funzioni già per input più piccoli.

La ricorsione lineare si divide in tre sottotipi a seconda di dove avviene il lavoro rispetto alla chiamata ricorsiva.

2.1 Post-order

Il lavoro avviene **dopo** la chiamata ricorsiva. La chiamata ricorsiva viene fatta prima, il suo risultato viene usato dopo.

Esempio 3.1 (Somma elementi positivi).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** uso la funzione (corretta per ipotesi su $i+1$) e aggiungo il contributo di $v.at(i)$.

```
int contributo = v.at(i) > 0 ? v.at(i) : 0;
```

```
return contributo + somma(v, i + 1);
```

- **Caso base:** quando $i == (int)v.size()$ non ci sono più elementi da esaminare; la somma è 0.

```
if (i == (int)v.size()) return 0;
```

```
1 int somma(const vector<int>& v, int i = 0) {  
2     if (i == (int)v.size()) return 0;  
3     int contributo = v.at(i) > 0 ? v.at(i) : 0;  
4     return contributo + somma(v, i + 1);  
5 }  
6
```

Esempio 3.2 (Massimo).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** uso la funzione (corretta per ipotesi su $i+1$) e confronto $v.at(i)$ con il massimo del resto già noto.

```
return max(v.at(i), massimo(v, i + 1));
```

- **Caso base:** quando $i == (int)v.size() - 1$ c'è un solo elemento, il massimo è lui.

```
if (i == (int)v.size() - 1) return v.at(i);
```

```
1 int massimo(const vector<int>& v, int i = 0) {
2     if (i == (int)v.size() - 1) return v.at(i);
3     return max(v.at(i), massimo(v, i + 1));
4 }
5
```

2.2 Pre-order

Il lavoro avviene prima della chiamata ricorsiva. L'effetto è l'opposto della post-order: si elabora in ordine naturale invece che inverso.

Esempio 3.3 (Stampa da 1 a n).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** uso la funzione (corretta per ipotesi su $i+1$) e stampo i prima di delegare il resto, il lavoro avviene alla discesa (pre-order).

```
cout << i << " ";
```

```
stampa(i + 1, n);
```

- **Caso base:** quando $i > n$ non c'è più nulla da stampare.

```
if (i > n) return;
```

```
1 void stampa(int i, int n) {
2     if (i > n) return;
3     cout << i << " ";
4     stampa(i + 1, n);
5 }
6
```

Esempio 3.4 (Somma con accumulatore).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo, con **acc** che porta la somma parziale degli elementi positivi già visitati.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** uso la funzione (corretta per ipotesi su $i+1$) aggiornando l'accumulatore con il contributo dell'elemento corrente nessun calcolo rimane pendente al ritorno.

```
int contributo = v.at(i) > 0 ? v.at(i) : 0;
```

```
return somma(v, i + 1, acc + contributo);
```

- **Caso base:** quando $i == v.size()$ tutti gli elementi sono stati processati; viene restituito direttamente l'accumulatore **acc**.

```
if (i == v.size()) return acc;
```

```
1 int somma(const vector<int>& v, int i = 0, int acc = 0) {
2     if (i == v.size()) return acc;
3     int contributo = v.at(i) > 0 ? v.at(i) : 0;
4     return somma(v, i + 1, acc + contributo);
5 }
6
```

2.3 Tail recursion

La chiamata ricorsiva è l'ultima operazione assoluta della funzione nessun calcolo pendente dopo di essa. Lo stato che nella post-order si accumula nello stack viene qui portato in avanti come parametro accumulatore.

Esempio 3.5 (Somma con accumulatore).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo, con **acc** che porta la somma parziale degli elementi positivi già visitati.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** uso la funzione (corretta per ipotesi su $i+1$) aggiornando l'accumulatore con il contributo dell'elemento corrente nessun calcolo rimane pendente al ritorno.

```
int contributo = v.at(i) > 0 ? v.at(i) : 0;
```

```
return somma(v, i + 1, acc + contributo);
```

- **Caso base:** quando $i == (int)v.size()$ tutti gli elementi sono stati processati; viene restituito direttamente l'accumulatore **acc** .

```
if (i == (int)v.size()) return acc;
```

```
1  int somma(const vector<int>& v, int i = 0, int acc = 0) {
2      if (i == (int)v.size()) return acc;
3      int contributo = v.at(i) > 0 ? v.at(i) : 0;
4      return somma(v, i + 1, acc + contributo);
5  }
6
```

Esempio 3.6 (Massimo comun divisore di Euclide).

La proprietà matematica su cui si fonda l'algoritmo è la seguente:

$$MCD(a, b) = MCD(b, a \bmod b)$$

Perché è vera. Dalla divisione euclidea si ha $a = q \cdot b + r$ con $r = a \bmod b$. Qualsiasi divisore comune d di a e b divide anche $r = a - q \cdot b$, e viceversa. I due insiemi di divisori comuni di (a, b) e (b, r) sono quindi identici, e in particolare lo è il loro massimo.

Metodologia risolutiva proposta:

- **Ricorsione su:** b , ridotto via $a \bmod b$ ad ogni passo. Poiché $0 \leq a \bmod b < b$, il secondo argomento decresce strettamente la sequenza raggiunge $b = 0$ in tempo finito.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi valore di b strettamente minore di quello corrente.
- **Passo ricorsivo:** per la proprietà sopra, $MCD(a, b) = MCD(b, a \bmod b)$; per ipotesi la funzione è già corretta su $(b, a \bmod b)$, quindi la chiamata ricorsiva restituisce il risultato corretto. La chiamata è tail: $a \bmod b$ viene calcolato prima di essere passato, nessun calcolo rimane pendente al ritorno.
- **Caso base:** quando $b = 0$, ogni intero divide 0, quindi $MCD(a, 0) = a$.

```
1 int mcd(int a, int b) {  
2     if (b == 0) return a;  
3     return mcd(b, a % b);  
4 }  
5
```

Ora andiamo a vedere il secondo pattern la **Tail recursion**.

3 Tail recursion

Definizione 3.1: Tail recursion

Una chiamata ricorsiva è in coda, quando è l'ultima operazione assoluta della funzione nessun calcolo rimane pendente dopo di essa. Il valore restituito della chiamata ricorsiva viene restituito direttamente, senza ulteriori elaborazioni.

La tail recursion si divide in due famiglie principali: con accumulatore e senza accumulatore:

3.1 Senza accumulatore

La chiamata ricorsiva è l'ultima operazione, ma non c'è nulla da comunicare in avanti, il risultato della chiamata ricorsiva viene restituito direttamente senza modifiche.

Si usa quando la ricorsione **elimina** possibilità invece di costruire qualcosa. Tipicamente ricerche, verifiche, riduzioni immediate.

Esempio 3.7 (Ricerca lineare).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** se $v.at(i)$ non è il target, uso la funzione (corretta per ipotesi su $i+1$) e ne restituisco direttamente il risultato nessun calcolo rimane pendente al ritorno.

```
return contiene(v, target, i + 1);
```

- **Casi base:** due condizioni distinte.

```
if (i == (int)v.size()) return false;
```

```
if (v.at(i) == target) return true;
```

```

1  bool contiene(const vector<int>& v, int target, int i = 0)
    {
2      if (i == (int)v.size()) return false;
3      if (v.at(i) == target) return true;
4      return contiene(v, target, i + 1);
5  }
6
```

Esempio 3.8 (Vettore ordinato).**Metodologia risolutiva proposta:**

- Facciamo la **ricorsione sull'indice i** , avanzato di 1 ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di i .
- **Passo ricorsivo:** se $v.at(i) \leq v.at(i+1)$, il confronto corrente è soddisfatto; uso la funzione (corretta per ipotesi su $i+1$) e ne restituisco direttamente il risultato nessun calcolo rimane pendente al ritorno.

```
return ordinato(v, i + 1);
```

- **Casi base:** due condizioni distinte.

```
if (v.at(i) > v.at(i + 1)) return false;
```

```
if (i == (int)v.size() - 1) return true;
```



```

1  bool ordinato(const vector<int>& v, int i = 0) {
2      if (i == (int)v.size() - 1) return true;
3      if (v.at(i) > v.at(i + 1)) return false;
4      return ordinato(v, i + 1);
5  }
6

```

3.2 Con accumulatore

Serve quando si deve costruire o calcolare qualcosa lungo il percorso. Questo sottotipo a sua volta ha altri sottotipi:

Accumulatore singolo numerico

Lo stato è un singolo valore numerico che si aggiorna ad ogni passo.

Esempio 3.9 (Fattoriale con accumulatore).

Il **fattoriale** di un intero non-negativo n ammette una definizione ricorsiva diretta:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \cdot (n - 1)! & \text{se } n > 0 \end{cases}$$

Per ottenere una **tail recursion**, si introduce un accumulatore **acc** che porta il prodotto parziale in avanti invece di lasciarlo pendente sullo stack. Poiché **acc** è un dettaglio implementativo interno, si separa la logica in due funzioni: una **funzione helper** che espone **acc** come parametro esplicito, e una **funzione wrapper** pubblica che la chiama inizializzando **acc** al valore neutro della moltiplicazione, ovvero **1**.

Metodologia risolutiva proposta:

- Facciamo la **ricorsione su n** , ridotto di 1 ad ogni passo, con accumulatore **acc** che porta il prodotto parziale.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi valore di **n** strettamente minore di quello corrente.
- **Passo ricorsivo:** aggiorno **acc** moltiplicando per **n** e delego nessun calcolo rimane pendente al ritorno.

```
return fact_helper(n - 1, acc * n);
```

- **Caso base:** quando **$n == 0$** il calcolo è completo; viene restituito direttamente **acc**.

```
if (n == 0) return acc;
```

```

1  int fact_helper(int n, long long acc) {
2      if (n == 0) return acc;
3      return fact_helper(n - 1, acc * n);
4  }
5
6  int fact(int n) {
7      return fact_helper(n, 1);    // acc inizializzato al
    valore neutro
8  }
9

```

3.3 Accumulatore stringa

Lo stato è una stringa che si costruisce progressivamente.

Esempio 3.10 (Inversione stringa con accumulatore).

Per ottenere una *tail recursion*, si introduce un accumulatore *acc* di tipo *string* che costruisce la stringa invertita progressivamente, portandola in avanti invece di lasciarla pendente sullo stack. *acc* viene inizializzato alla stringa vuota, valore neutro della concatenazione.

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice *i***, avanzato di 1 ad ogni passo, con accumulatore *acc* che porta la stringa invertita parziale.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi indice strettamente maggiore di *i*.
- **Passo ricorsivo:** prependo *s.at(i)* all'accumulatore e delego nessun calcolo rimane pendente al ritorno.

```
return inverti(s, i + 1, s.at(i) + acc);
```

- **Caso base:** quando *i == (int)s.size()* tutti i caratteri sono stati processati; viene restituito direttamente *acc*.

```
if (i == s.size()) return acc;
```

```

1  string inverti(const string& s, int i = 0, string acc = ""
    ) {
2      if (i == (int)s.size()) return acc;
3      return inverti(s, i + 1, s.at(i) + acc);
4  }
5

```

3.4 Accumulatori multipli paralleli

Serve quando lo stato richiede più valori che evolvono insieme. I parametri accumulatori sono due o più e si aggiornano in parallelo ad ogni passo.

Esempio 3.11 (Fibonacci con accumulatori paralleli).

La successione di Fibonacci è definita come:

$$F(n) = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ F(n-1) + F(n-2) & \text{se } n > 1 \end{cases}$$

La definizione standard genera una ricorsione multipla con sottoproblemi sovrapposti. Per ottenere una *tail recursion* si introducono due accumulatori paralleli **a** e **b** che portano avanti gli ultimi due valori della sequenza, aggiornandosi ad ogni passo senza calcoli pendenti al ritorno.

Metodologia risolutiva proposta:

- Facciamo la **ricorsione su *n***, ridotto di 1 ad ogni passo, con due accumulatori paralleli **a** e **b** che portano rispettivamente $F(k)$ e $F(k+1)$ per il passo corrente.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi valore di **n** strettamente minore di quello corrente.
- **Passo ricorsivo:** aggiorno entrambi gli accumulatori in parallelo **a** diventa **b** e **b** diventa **a+b** e delego. Nessun calcolo rimane pendente al ritorno.

return fib(n - 1, b, a + b);

- **Caso base:** quando **n == 0**, **a** contiene già $F(0)$; viene restituito direttamente.

if (n == 0) return a;

```

1  int fib_helper(int n, long long a, long long b) {
2      if (n == 0) return a;
3      return fib_helper(n - 1, b, a + b);
4  }
5
6  int fib(int n) {
7      return fib_helper(n, 0, 1);    // a=F(0)=0, b=F(1)=1
8  }
9

```

4 Divide et impera

Definizione 4.1

Divide et impera è una strategia ricorsiva che risolve un problema dividendolo in sotto problemi indipendenti della stessa natura, risolvendo ognuno per ipotesi induttiva, e poi combinando i risultati parziali in una soluzione completa.

Nel pattern Divide et impera ci sono due sottotipi, i quali sono:

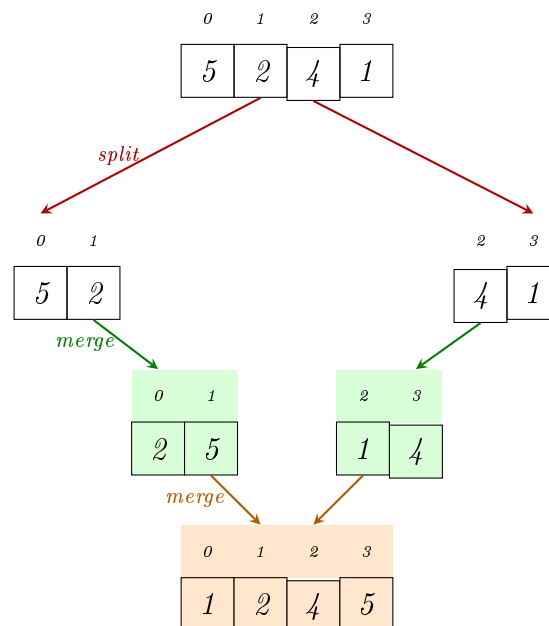
4.1 Divide et impera con combine esplicita

Il lavoro principale avviene nella fase combina, dopo che entrambe la metà sono state risolte, si fa un lavoro non banale per unire i risultati.

Esempio 3.12 (Merge sort).

Il **Merge Sort** è un algoritmo di ordinamento divide et impera. Opera in due fasi:

1. **Divide:** trova il punto medio $m = \lfloor (l+r)/2 \rfloor$ e richiama ricorsivamente **mergeSort** sulle due metà $[l, m]$ e $[m+1, r]$.
2. **Merge:** fonde le due metà ordinate confrontando elemento per elemento e copiando ogni volta il minore in un array di supporto, poi ricopia il risultato in **arr**[$l..r$].



Metodologia risolutiva proposta:

- Facciamo la **ricorsione sugli estremi l e r** dimezzando l'intervallo ad ogni passo.
- **Ipotesi induttiva::** la funzione è corretta per qualsiasi intervallo di dimensione strettamente minore di $r-l+1$
- **Passo ricorsivo:** per ipotesi le due metà sono già ordinate.
- **Caso base:** quando $l \geq r$ c'è un solo elemento ed è già ordinato

```
1 void merge(vector<int>& v, int l, int m, int r) {
2     vector<int> sx, dx;
3
4     for (int i = l; i <= m; i++) sx.push_back(v.at(i));
5     for (int i = m + 1; i <= r; i++) dx.push_back(v.at(i))
6     ;
7
8     int i = 0, j = 0, k = l;
9     while (i < (int)sx.size() && j < (int)dx.size())
10         v.at(k++) = (sx.at(i) <= dx.at(j)) ? sx.at(i++) :
11         dx.at(j++);
12     while (i < (int)sx.size()) v.at(k++) = sx.at(i++);
13     while (j < (int)dx.size()) v.at(k++) = dx.at(j++);
14 }
```

```
1 void mergeSort(vector<int>& v, int l, int r) {
2     if (l >= r) return;
3     int m = l + (r - l) / 2; // DIVIDE
4     mergeSort(v, l, m); // CONQUER sinistra
5     mergeSort(v, m + 1, r); // CONQUER destra
6     merge(v, l, m, r); // COMBINE
7 }
8
```

Esempio 3.13 (Massimo).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione** sugli estremi l e r , dimezzando l'intervallo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi intervallo di dimensione strettamente minore di $r-l+1$.
- **Passo ricorsivo:** per ipotesi ho il massimo delle due metà; prendo il maggiore tra i due.
- **Caso base:** quando $l==r$ c'è un solo elemento

```

1     int massimo(const vector<int>& v, int l, int r){
2     if(l==r) return v.at(l);
3     int m = l+(r-1)/2;
4     int maxSx = massimo(v, l, m);           // CONQUER
sinistra
5     int maxDx = massimo(v, m + 1, r);       // CONQUER destra
6     return max(maxSx, maxDx);               // Combina
7     }
8

```

4.2 Divide et impera senza combina, con riduzione diretta

Non c'è fase di combine dopo aver diviso, si procede su una sola delle parti. È un caso degenero di Divide et impera che assomiglia alla ricorsione lineare, ma il criterio di divisione è $n/2$ invece di $n-1$.

Esempio 3.14 (Fast power).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione** su exp , dimezzato ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi valore di exp strettamente minore di quello corrente
- **Passo ricorsivo:** per ipotesi ho $base^{\hat{exp}/2}$.
- **Caso base:** $exp == 0$

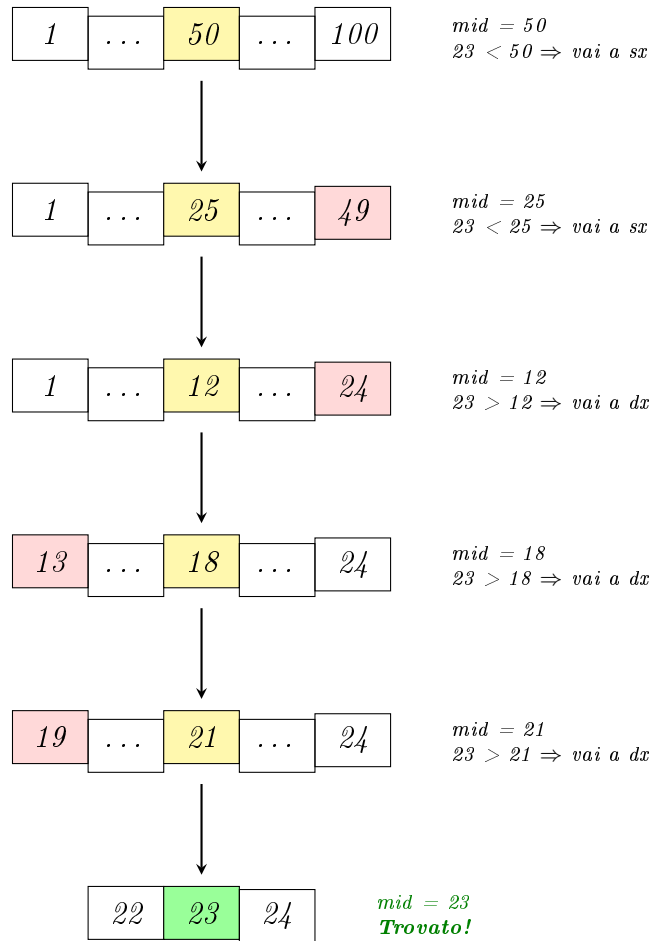
```
1      int fastPow(int base,int exp){
2          if(exp== 0) return 1;
3          int half = fastPow(base,exp/2);
4          if(exp%2 == 0) return half *half;
5          else
6              return base*half*half;
7      }
8
```

Esempio 3.15.

Supponiamo che qualcuno pensi a un numero tra 1 e 100, e noi dobbiamo indovinarlo. Ci verrà detto solo se è troppo alto, troppo basso, o se il numero è esatto. Per fare ciò ci sono due strategie:

- **Strategia non ottimale:** Proviamo 1, poi 2, poi 3... se il numero fosse 99, ci vorrebbero 99 tentativi. Ogni tentativo elimina un solo numero.
- **Strategia ottimale:** Ad ogni tentativo puntiamo sempre al centro dell'intervallo:
 1. Proviamo 50 → troppo alto elimino i numeri da 50 a 100.
 2. Proviamo 25 → troppo alto elimino i numeri da 25 a 49.
 3. Proviamo 12 → troppo basso limino i numeri da 1 a 12 .
 4. Proviamo 18 → troppo basso limino i numeri da 13 a 18.
 5. Proviamo 21 → troppo basso limino i numeri da 19 a 21.
 6. 23 trovato.

Ricerca Binaria — Cerco il 23 in [1, 100]



mid = elemento centrale **scartato** **trovato!**


```
1      bool ricercaBinaria(const vector<int>& v, int target,
2      int da, int a){
3          if(da>=a){
4              int m = (da+a)/2
5              if(v.at(m)==2)
6                  return true;
7          else
8              if(v.at(m)>e)
9                  return ricercaBinaria(v,e,da,m-1);
10             else
11                 return ricercaBinaria(v,e,m+1,a);
12         }
13     else
14         return false;
15 }
```

5 Backtracking

Definizione 5.1: Backtracking

Il backtracking è una tecnica ricorsiva per esplorare sistematicamente uno spazio di scelte costruendo una soluzione incrementalmente. Ad ogni passo si effettua una scelta, si assume per ipotesi induttiva che la ricorsione esplori correttamente tutto ciò che segue, poi si annulla la scelta per esplorare le alternative.

Nel backtracking ci sono tre sottotipi i quali sono:

5.1 Backtracking su scelte binarie

Ad ogni passo la scelta è binaria: includo l'elemento corrente oppure no.

Esempio 3.16 (sottoinsiemi).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sull'indice i** , avanzando ad ogni passo.
- **Passo ricorsivo:** per ogni elemento $v.at(i)$ esploro i due rami — incluso e non incluso — usando la funzione corretta per ipotesi su $i+1$.
- **Caso base:** $i == v.size()$ la soluzione parziale è completa, e viene registrata.

```

1   void subsets(const vector<int> & v, int i, vector<int> &
corrente, vector<vector<int>> & res){
2   if(i== v.size()){
3       res.push_back(corrente);
4       return;
5   }
6   subsets(v,i+1,corrente,res);
7   corrente.push_back(v.at(i));
8   subsets(v, i + 1, corrente, res);
9   corrente.pop_back();
10  }
11

```

5.2 Backtracking su permutazioni

Ad ogni passo le scelte sono filtrate da vincoli che dipendono dallo stato corrente.

Esempio 3.17 (Permutazioni).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sulla lunghezza di `corrente`**, aumentata di 1 ad ogni passo.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi stato con un elemento in più già scelto in `corrente`.
- **Passo ricorsivo:** per ogni elemento non ancora usato, lo aggiungo a `corrente`, uso la funzione (corretta per ipotesi) per esplorare il resto, poi annullo la scelta per provare le alternative.

```

usato.at(i) = true;

corrente.push_back(v.at(i));

permutazioni(v, usato, corrente, res);

corrente.pop_back();

usato.at(i) = false;

```

- **Caso base:** quando `(int)corrente.size() == (int)v.size()` la permutazione è completa; viene registrata.

```

if ((int)corrente.size() == (int)v.size())
res.push_back(corrente);

```

```
1 void permutazioni(const vector<int>& v, vector<bool>&
2   usato,
3   vector<int>& corrente, vector<vector<int
4   >>& res) {
5   if ((int)corrente.size() == (int)v.size()) {
6     res.push_back(corrente);
7     return;
8   }
9   for (int i = 0; i < (int)v.size(); i++) {
10    if (!usato.at(i)) {
11      usato.at(i) = true;
12      corrente.push_back(v.at(i));
13      permutazioni(v, usato, corrente, res);
14      corrente.pop_back();
15      usato.at(i) = false;
16    }
17  }
```

5.3 Backtracking con vincoli strutturali

Ad ogni passo le scelte sono filtrate da vincoli che dipendono dallo stato corrente.

Esempio 3.18 (Parentesi ben formate).

Metodologia risolutiva proposta:

- Facciamo la **ricorsione sulla lunghezza di `corrente`**, aumentata di 1 ad ogni passo, con contatori **`open`** e **`close`** che tracciano le parentesi già inserite.
- **Ipotesi induttiva:** la funzione è corretta per qualsiasi stato con un carattere in più già aggiunto a **`corrente`**.
- **Passo ricorsivo:** provo ad aggiungere (se **`open < n`**, e) se **`close < open`**; il pruning è implicito nelle condizioni. L'undo è implicito poiché **`corrente`** è passata per valore.

```
if (open < n)
```

```
    parentesi(n, open+1, close, corrente+"(", res);
```

```
if (close < open)
```

```
    parentesi(n, open, close+1, corrente+")", res);
```

- **Caso base:** quando **`(int)corrente.size() == 2*n`** la stringa è completa; viene registrata.

```
if ((int)corrente.size() == 2*n) res.push_back(corrente);
```

```

1 void parentesi(int n, int open, int close,
2               string corrente, vector<string>& res) {
3     if ((int)corrente.size() == 2 * n) {
4         res.push_back(corrente);
5         return;
6     }
7     if (open < n)
8         parentesi(n, open + 1, close, corrente + "(", res)
9     ;
10    if (close < open)
11        parentesi(n, open, close + 1, corrente + ")", res)
12    ;
13 }
```

CAPITOLO 4

CLASSI

1	Introduzione alle classi	62
1.1	Metodi	62
1.2	Struttura generale di una classe	62
1.3	Implementare i metodi fuori dalla classe	62
1.4	Incapsulamento	62
1.5	Coordinate	63
1.6	Implementazione dei metodi della classe Punto	63
1.7	costruttore	64
1.8	Distruttore	65
2	Live Coding Classi in C++	65

1 Introduzione alle classi

Una struct o classe permette di creare un tipo di dato personalizzato che raggruppa più informazioni insieme. Ad esempio, una `struct Prova` può contenere tre campi; un `float x`, un `int y` e una `string s`. La cosa importante da capire è che la combinazione dei valori di tutti i campi in un dato momento rappresenta lo stato dell'oggetto.

```
1      struct Punto{  
2          float x;  
3          int y;  
4          string s;  
5      }  
6
```

1.1 Metodi

Oltre ai campi, una `struct` può avere dei metodi, che sono semplicemente funzioni definite al suo interno. Ad esempio, `void stampa()` è un metodo che stampa "hello world" a schermo. La distinzione è netta: i campi descrivono cosa è l'oggetto, i metodi descrivono cosa sa fare.

Una volta definita la `struct`, si possono istanziare (creare) oggetti di quel tipo, ad esempio con Prova `o1`, `o2`;. Ogni oggetto è indipendente: `o1.x = 23.12` e `o2.x = 14.33` assegnano valori diversi allo stesso campo su istanze diverse. Per accedere a campi o metodi si usa l'operatore punto (`.`), come in `o1.stampa()`.

1.2 Struttura generale di una classe

Una classe può avere tre livelli di visibilità: `public`, `private` e `protected`. Nella sezione `public` si mettono i metodi che l'utente della classe può usare dall'esterno. Nella sezione `private` si mettono sia i dati che eventuali metodi interni che non devono essere esposti. Quando un metodo viene definito direttamente dentro la classe non devono essere esposti. Quando un metodo viene definito direttamente dentro la classe si dice che è **inline**: dichiarazione e definizione coincidono nello stesso punto.

1.3 Implementare i metodi fuori dalla classe

Nelle classi i metodi si dichiarano dentro la classe e si definiscono separatamente usando l'operatore `::`.

1.4 Incapsulamento

I campi `double x` e `double y` vengono dichiarati private, il che significa che non possono essere modificati direttamente dall'esterno della `struct`. Per farlo, si devono usare i metodi `public` come `setx()` e `sety()`. In questo modo si nascondono i dati interni e si controlla come vengono modificati, evitando usi scorretti.

```
1      struct Punto{
2          private:
3              double x;
4              double y;
5          public:
6              void setx(double nx);
7              void sety(double ny);
8              void setm(double nm);
9              void setdg(double na);
10             void stampa();
11     }
12
```

1.5 Coordinate

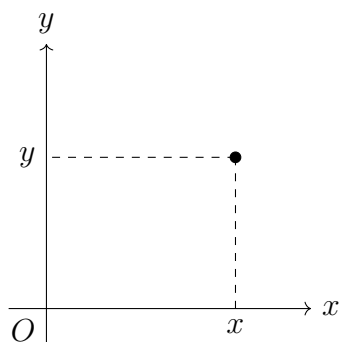
Un punto nel piano può essere rappresentato in due modi matematicamente equivalenti:

- **Cartesiano:** con (x, y) , le due coordinate lungo gli assi.
- **Polare:** con (ρ, θ) , dove ρ è la distanza dall'origine e θ è l'angolo rispetto all'asse x .

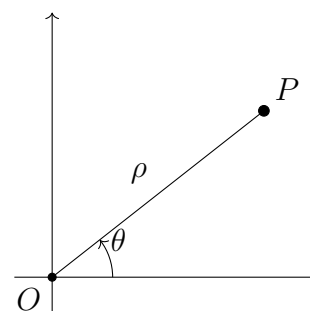
La `struct` `Punto` sfrutta proprio questa dualità: internamente salva sempre x e y come campi privati, ma espone anche i metodi `setm()` (per impostare il modulo ρ) e `setdg()`. Questi metodi eseguono internamente la conversione matematica:

$$x = \rho \cos \theta \quad y = \rho \sin \theta$$

Chi usa la classe non deve sapere come i dati sono salvati internamente può usare indifferentemente il sistema cartesiano o quello polare, e sarà a gestire la conversione.



Coordinate cartesiane



Coordinate polari

Figura 4.1: Rappresentazione di un punto nel piano

1.6 Implementazione dei metodi della classe `Punto`

Ora vedremo l'implementazione dei metodi della classe `Punto`:

```

1      void punto:: setx(double nx){
2          x=nx;
3      }
4

```

```

1      void punto:: sety(double ny){
2          y=ny;
3      }
4

```

```

1      void Punto::setrm(double nm) {
2          double theta = atan2(y, x);      // recupera l'angolo
corrente
3          x = nm * cos(theta);
4          y = nm * sin(theta);
5      }
6

```

Il metodo `setrm(double nm)` è un esempio di come l'incapsulamento protegga la coerenza interna dell'oggetto. L'obiettivo è cambiare il modulo ρ mantenendo invariato l'angolo θ : il punto si sposta lungo la stessa direzione ma a distanza diversa dall'origine. Senza incapsulamento, un utente potrebbe cambiare x senza aggiornare y , rompendo la coerenza con i metodi privati questo è impossibile.

1.7 costruttore

Il costruttore è un metodo che viene chiamato automaticamente ogni volta che si crea un nuovo oggetto di una classe. Ha due caratteristiche che lo distinguono da tutti gli altri metodi: porta lo stesso nome della classe e non ha tipo di ritorno. Il suo scopo è garantire che l'oggetto nasca in uno stato valido, evitando che i campi contengano valori casuali presenti in memoria

```

1      Punto::Punto() {
2          x = 0;
3          y = 0;
4      }

```


1.8 Distruttore

Il distruttore è un metodo che viene chiamato automaticamente quando un oggetto viene eliminato, sia che esca dallo scope, sia che venga cancellato esplicitamente con `delete`. La sua sintassi è identica a quella del costruttore, ma con una tilde `~` davanti al nome della classe, e anche lui non ha tipo di ritorno né parametri:

```
1  Punto::~ ~ Punto() {  
2  }
```

2 Live Coding Classi in C++

In questa lezione verrà svolta una sessione di live coding dedicata all'introduzione delle classi in C++. Durante la lezione verranno analizzati i concetti fondamentali legati alla definizione di una classe, alla distinzione tra sezione `public` e `private`, e alla gestione della memoria dinamica mediante costruttore, distruttore e copy constructor.

L'obiettivo è mostrare in modo pratico il ciclo di vita di un oggetto, la differenza tra allocazione su stack e su heap, e il motivo per cui il passaggio per valore di oggetti che gestiscono memoria dinamica può portare a errori come il *segmentation fault*.

Il materiale completo della lezione, insieme al notebook e al codice sviluppato durante la sessione, sarà disponibile nella repository del corso:

Repository: <https://github.com/ibrahimatellybarry23/pelmod1/tree/main/LiveCoding>